

```

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.Executors;
import java.util.concurrent.ScheduledExecutorService;
import java.util.concurrent.TimeUnit;

// =====
// VPN Health Monitor - Observer Pattern Implementation
// According to UML Diagram
// MUHAMMAD SAMUDERA BAGJA
// =====

public class VpnHealthMonitorApp {

    // =====
    // DATA TRANSFER OBJECT (immutable according to best practice)
    // =====
    static final class VpnHealthContext {
        private final String nodeId;
        private final String status;
        private final int pingMs;
        private final LocalDateTime timestamp;
        private final boolean isDown;

        public VpnHealthContext(String nodeId, String status, int pingMs,
                                LocalDateTime timestamp, boolean isDown) {
            this.nodeId = nodeId;
            this.status = status;
            this.pingMs = pingMs;
            this.timestamp = timestamp;
            this.isDown = isDown;
        }

        public String getNodeId() {
            return nodeId;
        }

        public String getStatus() {
            return status;
        }

        public int getPingMs() {
            return pingMs;
        }

        public LocalDateTime getTimestamp() {
            return timestamp;
        }

        public boolean isDown() {
            return isDown;
        }
    }

```

```

@Override
public String toString() {
    return String.format("[%s] Node: %s | Status: %s | Ping: %d ms | Down:...
        timestamp.format(DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm...
        nodeId, status, pingMs, isDown ? "YES" : "NO");
}
}

// =====
// OBSERVER INTERFACE
// =====
interface Subscriber {
    void update(VpnHealthContext context);
}

// =====
// SUBJECT – VpnHealthMonitor
// =====
static class VpnHealthMonitor {
    private final List<Subscriber> subscribers = new ArrayList<>();
    private String vpnStatus = "UNKNOWN";
    private LocalDateTime lastChecked = null;

    // --- Observer management ---
    public void subscribe(Subscriber s) {
        subscribers.add(s);
        System.out.println("[Monitor] Subscriber registered: " + s.getClass().....
    }

    public void unsubscribe(Subscriber s) {
        subscribers.remove(s);
        System.out.println("[Monitor] Subscriber removed: " + s.getClass().get...
    }

    private void notifySubscribers(VpnHealthContext ctx) {
        for (Subscriber s : subscribers) {
            try {
                s.update(ctx);
            } catch (Exception e) {
                System.err.println("[Monitor] Error notifying " +
                    s.getClass().getSimpleName() + ": " + e.getMessage());
            }
        }
    }
}

// --- Core logic ---
public void checkHealth() {
    lastChecked = LocalDateTime.now();

    // Simulate VPN check (replace with real logic)
    int pingMs = simulatePing();
    boolean isDown = pingMs < 0 || pingMs > 500;
    vpnStatus = isDown ? "DOWN" : "UP";
}

```

```

        VpnHealthContext ctx = new VpnHealthContext(
            "vpn-node-01", vpnStatus, Math.max(pingMs, 0), lastChecked, is...

        System.out.println("\n[Monitor] Health check: " + ctx);
        notifySubscribers(ctx);
    }

    public String getStatus() {
        return vpnStatus;
    }

    // Simulate ping (returns -1 to simulate DOWN state)
    private int simulatePing() {
        double r = Math.random();
        if (r < 0.2)
            return -1; // 20% chance: VPN down
        if (r < 0.4)
            return 450; // 20% chance: high latency
        return (int) (r * 100) + 10; // normal
    }
}

// =====
// CONCRETE OBSERVER 1 – TelegramAlertListener
// =====
static class TelegramAlertListener implements Subscriber {
    private final String botToken;
    private final String chatId;
    private final String alertMessage;

    public TelegramAlertListener(String botToken, String chatId, String alertM...
        this.botToken = botToken;
        this.chatId = chatId;
        this.alertMessage = alertMessage;
    }

    @Override
    public void update(VpnHealthContext context) {
        if (context.isDown()) {
            String msg = String.format("%s\nNode: %s\nStatus: %s\nTime: %s",
                alertMessage,
                context.getNodeId(),
                context.getStatus(),
                context.getTimestamp().format(DateTimeFormatter.ofPattern(...
            sendTelegramMessage(msg);
        }
    }

    private void sendTelegramMessage(String msg) {
        // Simulation – replace with HTTP call to Telegram Bot API
        System.out.println("[TelegramAlert] Sending to chat " + chatId + ":");
        System.out.println("  >> " + msg.replace("\n", " | "));
        // Real example:

```

```

        // HttpClient.newHttpClient().send(HttpRequest.newBuilder()
        // .uri(URI.create("https://api.telegram.org/bot" + botToken + "/sendM...
        // .POST(BodyPublishers.ofString("{\"chat_id\":\"" + chatId + "\",\"te...
        // msg + "\"}"))
        // .header("Content-Type", "application/json").build(),
        // BodyHandlers.ofString());
    }
}

// =====
// CONCRETE OBSERVER 2 – AutoRestartListener
// =====
static class AutoRestartListener implements Subscriber {
    private final String dockerServiceName;
    private final int maxRetries;
    private int retryCount = 0;

    public AutoRestartListener(String dockerServiceName, int maxRetries) {
        this.dockerServiceName = dockerServiceName;
        this.maxRetries = maxRetries;
    }

    @Override
    public void update(VpnHealthContext context) {
        if (context.isDown()) {
            if (retryCount < maxRetries) {
                restartDockerService();
                retryCount++;
            } else {
                System.out.println("[AutoRestart] Max retries (" + maxRetries +
                    ") reached for service: " + dockerServiceName + ". Ski...
            }
        } else {
            resetRetryCount();
        }
    }

    private void restartDockerService() {
        System.out.println("[AutoRestart] Restarting Docker service: " +
            dockerServiceName + " (attempt " + (retryCount + 1) + "/" + ma...
        // Real example:
        // Runtime.getRuntime().exec("docker restart " + dockerServiceName);
    }

    public void resetRetryCount() {
        if (retryCount > 0) {
            System.out.println("[AutoRestart] Service recovered. Resetting ret...
            retryCount = 0;
        }
    }
}

// =====
// DATABASE CONNECTION SIMULATION

```

```

// (Should be an interface – this is a simulation to run directly)
// =====
static class DBConnection {
    private final String url;

    public DBConnection(String url) {
        this.url = url;
        System.out.println("[DBConnection] Connected to: " + url);
    }

    public void executeUpdate(String sql, Object... params) {
        // Query simulation
        System.out.println("[DB] SQL: " + sql + " | Params: " + java.util.Arra...
    }
}

// =====
// CONCRETE OBSERVER 3 – DatabaseLogListener
// =====
static class DatabaseLogListener implements Subscriber {
    private final DBConnection dbConnection;
    private final String tableName;

    public DatabaseLogListener(DBConnection dbConnection, String tableName) {
        this.dbConnection = dbConnection;
        this.tableName = tableName;
    }

    @Override
    public void update(VpnHealthContext context) {
        insertLog(context);
        updateUptimeRecord(context.getNodeId());
    }

    private void insertLog(VpnHealthContext context) {
        String sql = String.format(
            "INSERT INTO %s (node_id, status, ping_ms, timestamp, is_down)...
            tableName);
        dbConnection.executeUpdate(sql,
            context.getNodeId(),
            context.getStatus(),
            context.getPingMs(),
            context.getTimestamp().toString(),
            context.isDown());
    }

    private void updateUptimeRecord(String nodeId) {
        String sql = "UPDATE uptime_records SET last_seen = NOW() WHERE node_i...
        dbConnection.executeUpdate(sql, nodeId);
    }
}

// =====
// APPLICATION ENTRY POINT

```

```
// =====
static class Application {
    private VpnHealthMonitor monitor;
    private ScheduledExecutorService scheduler;

    public void configure() {
        // 1. Create subject
        monitor = new VpnHealthMonitor();

        // 2. Create and register subscribers
        Subscriber telegramListener = new TelegramAlertListener(
            " Y O U R _ B O T _ T O K E N " ,    " Y O U R _ C H

        Subscriber autoRestart = new AutoRestartListener(
            "vpn-service", 3);

        DBConnection db = new DBConnection("jdbc:postgresql://localhost:5432/v...
        Subscriber dbLogger = new DatabaseLogListener(db, "vpn_health_logs");

        monitor.subscribe(telegramListener);
        monitor.subscribe(autoRestart);
        monitor.subscribe(dbLogger);

        // 3. Schedule health check every 10 seconds
        scheduler = Executors.newSingleThreadScheduledExecutor();
    }

    public void main() {
        System.out.println("=== VPN Health Monitor Started ===");
        configure();

        // Run 5 times for demo (2 seconds interval)
        scheduler.scheduleAtFixedRate(() -> {
            monitor.checkHealth();
        }, 0, 2, TimeUnit.SECONDS);

        // Stop after 12 seconds (demo)
        scheduler.schedule(() -> {
            System.out.println("\n=== Demo finished. Shutdown. ===");
            scheduler.shutdown();
        }, 12, TimeUnit.SECONDS);

        // Wait until finished
        try {
            scheduler.awaitTermination(15, TimeUnit.SECONDS);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}

// =====
// MAIN – Java entry point
// =====
```

```
public static void main(String[] args) {  
    new Application().main();  
}  
}
```